Agente de Inteligência Urbanística — Plano da v1

Nov 5, 2026 · @Marcos Yudi

Objetivo e escopo

A v1 transforma a lei de uso e ocupação do solo de Londrina em uma tabela consultável e mede o quanto a extração por LLM acerta. Prazo: MVP nesta semana + 4 semanas, cerca de 10 a 12 horas por semana.

O projeto também é a vitrine para o estágio GenAI 2027 do iFood, trilha AI Engineer. A inscrição fecha em Nov 17, 2026 e as etapas técnicas vêm em novembro e dezembro, por isso o MVP público vem primeiro, antes das entrevistas.

O que entra na v1

- Uma cidade: Londrina (confirmar no site da prefeitura qual é a lei vigente e seus anexos).
- Cinco parâmetros por zona: taxa de ocupação, coeficiente de aproveitamento, altura máxima ou gabarito, recuo frontal e taxa de permeabilidade.
- Pipeline completo: PDF bruto → extração estruturada → banco → consulta.
- Avaliação contra um gabarito feito à mão.
- Um demo publicado: dado uma zona, o app mostra o que pode ser construído.

O que fica de fora (vai para a v2)

- Outras cidades, mapas e geolocalização de lotes.
- Agente conversacional livre. Na v1 o agente só responde sobre os 5 parâmetros, consultando o banco.
- Exceções, usos permitidos por zona e regras de outorga.

Alinhamento com a vaga

A trilha AI Engineer "treina, ajusta, avalia e mantém modelos e pipelines", com ênfase em LLMs e desenvolvimento de software. Cada exigência tem uma prova no projeto.

O que a vaga pede	Onde o projeto mostra	Semana
Avaliar modelos	Gabarito manual + métricas de acurácia e alucinação	MVP
Manter modelos	Avaliação no CI que falha se a acurácia cair; prompts versionados	1
Ajustar modelos	Comparação de 2 a 3 LLMs por acurácia, custo e latência	1
Pipelines	PDF → Cloud Storage → extração → BigQuery, agendado	2
LLMs e agentes	Agente com tool calling que consulta o BigQuery e cita a lei	3
Desenvolvimento de software	API em FastAPI, Docker, testes e logs das chamadas ao LLM	3

História para as entrevistas: em 2 minutos, um erro que o modelo cometia, como você mediu, o que mudou e quanto a métrica subiu. Serve para a entrevista técnica e para o case.

Arquitetura

Do PDF ao demo, com a extração medida contra um gabarito



arquitetura da v1 · 8 componentes

O PDF vira dados estruturados no BigQuery. Dali saem o demo e a avaliação contra o gabarito, que é a parte que diferencia o projeto. O GitHub Actions roda tudo de forma automática.

Estrutura do repositório

Um repositório só seu, `agente-urbanistico`, separado por etapa do pipeline para que quem abrir entenda o fluxo pelas pastas.

```
agente-urbanistico/
├── README.md              # problema, arquitetura, métricas, comparação de
modelos, demo
├── pyproject.toml         # dependências (ou requirements.txt)
├── Dockerfile
├── .env.example           # variáveis sem segredos: projeto GCP, bucket,
chave da API
├── data/
│   ├── raw/               # PDFs baixados (fora do git, só no bucket)
│   └── gold/
```

```
|       └── gabarito.csv   # parâmetros preenchidos à mão
├── prompts/
|   ├── extracao_v1.md
|   └── extracao_v2.md     # cada versão com sua métrica no README
├── src/urbanistico/
|   ├── ingest.py          # baixa os PDFs e envia ao Cloud Storage
|   ├── parse.py           # PDF → texto e tabelas por página
|   ├── schema.py          # modelo Pydantic dos 5 parâmetros
|   ├── llm.py             # cliente dos modelos + logs de tokens, custo e
latência
|   ├── extract.py         # extração com saída JSON validada
|   ├── load.py            # grava no BigQuery
|   ├── evaluate.py        # compara com o gabarito e gera as métricas
|   └── agent.py           # agente com tool calling sobre o BigQuery
├── api/
|   └── main.py            # FastAPI: /extrair e /consultar
├── sql/
|   ├── create_tables.sql
|   └── consultas.sql      # perguntas de exemplo
├── app/
|   └── streamlit_app.py   # demo: escolhe a zona, vê os parâmetros
├── notebooks/
|   ├── 01_exploracao.ipynb
|   └── 02_comparacao_modelos.ipynb
├── tests/
|   └── test_schema.py
└── .github/workflows/
    ├── ci.yml             # lint + testes a cada push
    ├── eval.yml           # roda o gabarito e falha se a acurácia cair
    └── pipeline.yml       # roda o pipeline agendado
```

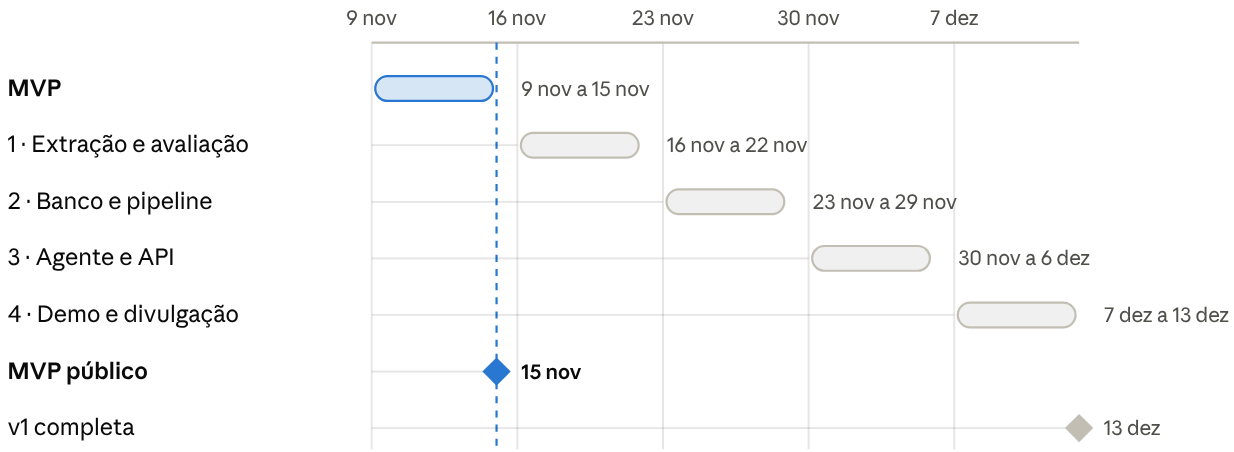
Regra de ouro: nenhuma chave de API ou senha no repositório. Use `.env` local e os *secrets* do GitHub.

Cronograma

O MVP fica público em `Nov 15, 2026` e a v1 completa termina em `Dec 13, 2026`. A semana do MVP pede de 15 a 18 horas; as quatro seguintes, cerca de 10 a 12 horas cada.

MVP público em 15/11, v1 completa em 13/12

Uma semana de MVP e quatro semanas de evolução, 2026



cronograma · MVP + 4 semanas

MVP — 9 a 15 de novembro (esta semana)

Inscrição no iFood já feita. Meta da semana: repositório público com extração e uma métrica de acurácia no README.

Dia	Tarefa	Entrega	Horas
Seg 9/11	Baixar a lei e os anexos; achar as tabelas de parâmetros; escolher 3 a 5 zonas	PDFs salvos e zonas definidas	2
Ter 10/11	Preencher `gabarito.csv` com a página de origem; criar o repositório e o `schema.py`	Gabarito com cerca de 25 valores; primeiro commit	3
Qua 11/11	`parse.py`: texto e tabelas das páginas certas; testar pdfplumber e PyMuPDF	Tabelas legíveis em Python	2 a 3
Qui 12/11	`extract.py` com o prompt v1: Gemini com saída JSON validada pelo Pydantic	JSON válido para as zonas do gabarito	3
Sex 13/11	`evaluate.py`: acurácia exata e por parâmetro	Primeira métrica (prompt v1)	2

Dia	Tarefa	Entrega	Horas
Sáb 14/11	Prompt v2 a partir dos erros; README com problema, como rodar e métricas v1 → v2	README com resultado	3 a 4
Dom 15/11	Revisar, fixar o repositório no GitHub e no README do perfil; folga para atrasos	**MVP público**	2

Semana 1 — 16 a 22 de novembro: extração robusta e avaliação contínua

- [ ] Ampliar o gabarito para 8 a 10 zonas (começar por ZR-3 e ZUM-2) e acrescentar as colunas afastamento lateral, afastamento de fundos, CA mínimo, lote mínimo e frente mínima (4 h).
- [ ] Mover os prompts para `prompts/`, com o histórico de métricas de cada versão (1 h).
- [ ] Guardar a página de origem de cada valor; valor ausente vira nulo (2 h).
- [ ] `eval.yml`: o GitHub Actions roda o gabarito e falha se a acurácia cair (2 h).
- [ ] `02_comparacao_modelos.ipynb`: 2 a 3 modelos comparados em acurácia, custo por documento e latência (3 h).

Semana 2 — 23 a 29 de novembro: banco e pipeline

- [ ] `load.py` + `create_tables.sql`: gravar no BigQuery.
- [ ] `evaluate.py`: salvar as métricas de cada execução no BigQuery.
- [ ] Ajustar o prompt uma ou duas vezes e registrar o antes e depois.
- [ ] `ingest.py`: enviar os PDFs ao Cloud Storage.
- [ ] `pipeline.yml`: rodar ingestão → extração → BigQuery de forma agendada.

Semana 3 — 30 de novembro a 6 de dezembro: agente e API

- [ ] `agent.py`: agente com tool calling que transforma a pergunta em consulta ao BigQuery e cita a página da lei (4 h).
- [ ] `api/main.py`: FastAPI com `/extrair` e `/consultar` (3 h).
- [ ] `Dockerfile` e testes da API e do schema (2 h).
- [ ] `llm.py`: logs de tokens, custo, latência e erros de cada chamada (2 h).

Semana 4 — 7 a 13 de dezembro: demo, acabamento e divulgação

- [ ] `streamlit_app.py`: escolher a zona e ver parâmetros e página de origem; publicar (3 h).
- [ ] README final com diagrama, métricas, comparação de modelos e GIF do demo (3 h).
- [ ] Cadastrar em Projetos no LinkedIn e publicar o post (2 h).
- [ ] Ensaiar a história de 2 minutos para as entrevistas (1 h).

Avaliação

A avaliação é o que transforma o projeto de "usei um LLM" em ciência de dados: um número que diz o quanto dá para confiar na extração.

O gabarito (`data/gold/gabarito.csv`) tem uma linha por zona e parâmetro: `zona, parametro, valor, unidade, pagina`. Preencha à mão antes de rodar o modelo, para não se deixar influenciar pelas respostas dele.

Métrica	O que mede	Como calcular
Acurácia exata	Valores idênticos ao gabarito	acertos ÷ total de valores
Acurácia por parâmetro	Qual campo o modelo erra mais	a mesma conta, separada por parâmetro
Taxa de nulos corretos	Se o modelo admite quando não encontra	nulos certos ÷ valores ausentes no gabarito
Taxa de alucinação	Números inventados	valores preenchidos onde o gabarito é nulo ÷ total

Registre cada versão do prompt e a métrica que ela teve. Uma tabela de "prompt v1 → v2 → v3" no README mostra seu processo de melhoria, que é exatamente o que recrutadores querem ver.

Critérios de pronto e divulgação

A v1 está pronta quando alguém de fora consegue entender, rodar e testar o projeto sem falar com você.

- [ ] O pipeline roda do PDF ao BigQuery com um comando.

- [ ] A métrica de acurácia está no README, com o gabarito publicado.
- [ ] O demo está no ar, com link no README.
- [ ] O CI passa e o pipeline agendado rodou pelo menos uma vez.
- [ ] O README explica o problema em 3 linhas, antes de qualquer detalhe técnico.

Divulgação

- GitHub: fixar o repositório em primeiro lugar e adicionar ao README do perfil como projeto principal.
- LinkedIn: cadastrar em Projetos e em Em destaque.
- Post: contar como arquiteto por que esses dados são difíceis de achar, mostrar o diagrama e o número de acurácia, e terminar com o link do demo. Siga o formato do post da Red Bull, que foi o que teve mais alcance.

Próximas versões

- **v2:** mais 2 ou 3 cidades do Paraná e uma análise comparando o quanto cada uma permite construir.
- **v3:** agente conversacional completo, com mais ferramentas (usos permitidos, outorga) e respostas sobre lotes específicos.